

RECURSION

[-2pt] **Key ideas:** base case (immediate, obvious); recursive case calls self on smaller/similar problem; each call has own local vars on stack; must approach base case. **Largest:** compare head vs. largest(rest). **Reverse:** remove head, recurse, append head at end. **Member-of:** head==x OR member(rest). **Multiply** i×j: base j=0; else i + mult(i,j-1).

TOWERS OF HANOI

[-2pt]
Move n disks src→dst using tmp. Recursive solution:
1. solve(n-1, src, dst, tmp) — move top n-1 via dst
2. move(src→dst) — base case: 1 disk
3. solve(n-1, tmp, src, dst) — move from tmp
Total moves: 2^n - 1. "Very difficult without recursion!" Pins swap roles (src/tmp/dst) each call.

FIBONACCI DISASTER

[-2pt] f(n) = f(n-1) + f(n-2): exponential redundant calls; bogs down. Use iteration instead. Recursion is powerful but use appropriately.

QUICKSORT

(1961, "most elegant")
Partition: pick middle as pivot (A); park at right by swap (B); set i=left-1, j=right; loop until j crosses i: move i right while data[i] (C); move j left while data[j]>pivot (D); if i swap data[i],data[j] (E); swap pivot back in at i. **Sort:** base: size<2 return; size=2 swap if needed; else partition, recursively sort left & right subarrays.

DYNAMIC BACKTRACKING

[-2pt]
Decision tree: try a path; backtrack on dead end; try next path. Base case = no further paths (dead end) or solution found. Can find all solutions by continuing after finding one.
N-Queens (8): solve column-by-column L-R. is_safe(position(row,col) checks left attacks. find_solutions() recurses right. After each safe square, backtrack to try next. **92 solutions.**
[-2pt] **Sudoku:** brute-force 1-9 per cell; L-R top-bottom; recursively check ahead; backtrack to next number after trying all 9; backtrack to previous cell.

MULTITHREADING

[-2pt] **Thread:** path through code; own stack+registers; shares code/data/resources. **Concurrency** (switch among tasks, 1 CPU) vs. **Parallelism** (simultaneous, multi-CPU). **Shared resource:** output stream, memory. **Critical region:** code accessing shared resource. **Mutual exclusion:** only 1 thread in critical region at a time. **mutex:** lock() blocks if already locked; unlock() on exit. **lock_guard< mutex>:** RAII — auto-locks on construct, auto-unlocks on scope exit; prevents deadlock from forgotten unlock. **Context switching:** runtime switches threads; this_thread::yield() voluntary relinquish. this_thread::sleep_for(chrono::seconds(n)). **Reader-Writer:** shared_mutex. Writers use unique_lock< shared_mutex> (exclusive — no readers or writers). Readers use shared_lock< shared_mutex> (multiple readers OK; blocks writers). **atomic< T>:** thread-safe without mutex; store(), load(), — ops. **condition_variable:** cv.wait(lock) — atomically unlocks+sleeps; re-locks when notified; always use while() loop (spurious wakeup). cv.notify_all() wakes all waiting.

PRODUCER-CONSUMER

[-2pt]
Queue with capacity. Producers wait on producer_cv when full (is_full()). Consumers wait on consumer_cv when empty (is_empty()). Producer notifies consumers when just_became_not_empty() (size==1). Consumer notifies producers when just_became_not_full() (size==CAP-1). When all producers done, notify consumers.
Dining Philosophers: 5 philosophers, 5 chopsticks (one between each). Need both adjacent; must pick up both at once (cannot hold one and wait). Loop: think-wait-fill bowl-eat. **Deadlock:** all blocked; prevent with consistent lock ordering. **auto:** type inference from initializer. **decltype(x):** yields type of variable x; use to declare same-type variable.

WELL-DESIGNED SOFTWARE

[-2pt]
Meets requirements · Reliable (no surprises, no hidden perf issues) · Flexible+scalable (add features without complexity) · Maintainable · Uses good design+patterns · Saves time/cost. Design-code-test iterations ("Mozart" analogy — iterative, not big-bang).

OOP DESIGN

[-2pt] **Encapsulation:** data+behavior in class; public/private; contains/isolates changes. **Abstraction:** ignore irrelevant details. **Inheritance:** subclass from superclass; adds/overrides state+behavior. **Polymorphism:** base ptr calls correct subclass method at runtime via virtual. **dynamic_cast< T*>(ptr):** type-check+convert; returns nullptr if wrong type. **Abstract class:** ≥1 pure virtual (=0); cannot instantiate; subclasses must implement. **Interface class:** all pure virtual, no member vars. **virtual destructor** required if subclasses exist. **Override** (same signature, different class) vs. **Overload** (same name, different params, same scope). **Liskov Substitution (LSP):** subclass substitutable for superclass. **Violation:** CircularBuffer:vector exposes invalid vector methods (at(), erase()); **Fix:** has-a not is-a (member vector buffer). **Code to Interface:** declare as base ptr, assign subclass at runtime.

DESIGN SMELLS

[-2pt] **Leaking changes:** change in Car forces change in Driver (key-button). **God class:** AutomobileApp — too many responsibilities (drive+maintain+wash). **Hardcoded concrete:** Cat *pet = new Cat(). **Principle of least surprise violation:** Date MONTH_NAMES[12] off-by-one (month=9-OCT not SEP). **DRY violation:** repeated code in multiple classes.

DESIGN PRINCIPLES (9)

[-2pt]
Principle SRP
p(1.2in){}
Key idea
One primary responsibility; cohesive; easy use+maintain
Encapsulate Isolate
What Varies changeable code; changes don't leak
Delegation Move work to suitable class;

requester no HOW dep.
Call only: member var, param, locally instantiated; NOT returned obj (a.getB().getC() is smell); return copies not ptrs
Closed to modification; open for extension via subclasses
Use base/interface ptr not specific subclass
No duplicate code; share as function/class
Runtime param checks; create only valid objects; assert()
Programming by Contract

Also: **Favor Composition over Inheritance** (has-a often better). **Lazy Evaluation.** **Refactoring** (improve without changing functionality).

REQUIREMENTS

[-2pt] **Functional:** what system shall() do / allow users to do. **Nonfunctional:** usability, reliability, performance, supportability — equally important. Use "shall"/"must" (strong); "should" = wish list. **Quality**
Clear
}
Meaning
Non-technical, jargon-free
Consistent No contradictions
Correct Each req must be correct
Complete No gaps
Realistic Satisfiable
Verifiable Testable
Traceable Each req ↔ feature

[-2pt]
Sources: interview, observe, stated+implied. **Iterative** — show prototypes; customers don't always know what they want. **I18N** (Internationalization, 18 letters) — design for multiple locales. **L10N** (Localization, 10 letters) — adapt language, dates, monetary, encoding. **V&V:** Validate=building right app (customer confirms); Verify=building app right (developers test).

UML DIAGRAMS

[-2pt] **Class:** class name; member vars (+pub/-priv/#prot); member fns. **6 relationships:** dependency, transient dependency, aggregation (has vector of), composition (part cannot exist outside container), generalization (superclass), realization (implements interface — dashed arrow). **Multiplicity** notation on arrows. **Use Case:** oval=use case; stick figure=actor (external agent: person or system); rectangle=system boundary. **State:** rounded boxes=states; arrows=transitions+events; filled circle=start; circle-in-circle=end; [conditions] in brackets. **Sequence:** lifelines (dashed vertical); active rectangles; labeled horizontal arrows=method calls; time top-bottom.

USE CASE DESCRIPTION (10 fields)

[-2pt] **Name** (verb-noun); **Goal; Summary** (1-2 sent.); **Actors; Preconditions** (what must be true before); **Trigger** (actor action to start); **Primary sequence** (≤10 steps); **Alternate sequences** (exception paths); **Postconditions; Nonfunctional requirements; Glossary.** Focus on WHAT not HOW; "black box" view; no GUI/impl details.

FUNCTIONAL SPEC & DESIGN SPEC

[-2pt] **Functional Spec** (External Ref Spec): product name, problem statement, objectives, functional reqs, nonfunctional reqs, use cases. No impl details. Customer reads/approves = **validates** (building right app). **Design Spec** (Internal Maintenance Spec): technical, impl details, UML class/state/sequence. Developers **verify** (building app right).

REQUIREMENTS ANALYSIS (textual)

[-2pt] **Nouns-Classes** (member vars for state). **Verbs-Behaviors/Methods** (transitive verbs acting on objects — member fns; assign cohesively per SRP). **Noun-Class Table:** noun, class, states/vars. **Verb-Behavior Table:** verb, class, member fn.

LAMBDA EXPRESSIONS